Relatório Técnico – DeliveryTech API

1. Visão Geral do Projeto

1.1 Objetivo Geral

A DeliveryTech API é uma aplicação **RESTful** desenvolvida com **Spring Boot 3.2.5** e **Java 21** para gerenciar um sistema de delivery de alimentos de ponta a ponta. O objetivo é oferecer uma base sólida, escalável e segura que simula plataformas como iFood e Uber Eats, cobrindo desde o cadastro de restaurantes até o fluxo completo de pedidos e entrega.

A API foi projetada para servir tanto como solução técnica para integrações reais quanto como projeto de estudo, demonstrando boas práticas de **arquitetura em camadas**, **segurança moderna com JWT**, **cache distribuído**, **observabilidade** e **automação de build/teste/deploy**.

1.2 Escopo

O escopo do sistema abrange:

- Cadastro e gestão de usuários, clientes, restaurantes, produtos e pedidos.
- Autenticação e autorização baseadas em **JWT**.
- Controle de acesso por perfis (roles):
- `CLIENTE`
- `RESTAURANTE`
- `ADMIN`
- `ENTREGADOR`
- Fluxo completo de pedidos:
- Criação de pedidos com itens.
- Cálculo automático de valores.
- Atualização de status ao longo do ciclo de vida.
- Cache de dados para otimização de performance.
- Observabilidade: métricas, health checks e tracing distribuído.
- Empacotamento em containers Docker e orquestração com Docker Compose.
- Suporte a múltiplos perfis de execução:
- `dev`: desenvolvimento local com H2.
- `test`: ambiente de testes automatizados.
- `prod`: ambiente de produção com Redis e banco relacional.

2. Público-Alvo

A solução atende diferentes perfis de usuários, tanto de negócio quanto técnicos:

- **Restaurantes**
  - Gerenciamento de cardápios.
  - Administração de pedidos (aceite, preparo, despacho).
  - Acompanhamento de métricas operacionais.
- **Clientes**
  - Cadastro de conta e autenticação.
  - Navegação por restaurantes e produtos.
  - Criação e acompanhamento de pedidos.
- **Administradores da Plataforma**
  - Gestão de usuários e perfis de acesso.
  - Governança de restaurantes e políticas da plataforma.
  - Monitoramento de saúde da aplicação e indicadores técnicos.
- **Desenvolvedores e Times Técnicos**
  - Integração da API a aplicativos web e mobile.
  - Extensão da solução com novas funcionalidades.
  - Estudos de referência em arquitetura Java/Spring moderna.

3. Funcionalidades Principais

3.1 Gestão de Usuários e Autenticação

- Cadastro de usuários com papéis específicos (`CLIENTE`, `RESTAURANTE`, `ADMIN`, `ENTREGADOR`).
- Autenticação via **JWT Bearer Token** com:
- Geração e assinatura de tokens.
- Validação de integridade e expiração.
- Extração de claims (ID do usuário, roles).
- Integração com o **Spring Security**:
- Filtros de autenticação JWT.
- Configuração de endpoints públicos e protegidos.
- Carregamento de detalhes do usuário a partir do banco de dados.

3.2 Gestão de Domínio (Clientes, Restaurantes, Produtos, Pedidos)

- **Clientes**
  - Cadastro, consulta e atualização de dados.
  - Associação de pedidos a clientes.
- **Restaurantes**
  - Cadastro de restaurantes.
  - Associação de produtos ao restaurante.
  - Exposição de cardápio.
- **Produtos**
  - Cadastro de produtos por restaurante.
  - Consulta de produtos por restaurante.
  - Informação de preço, descrição e disponibilidade.
- **Pedidos**
  - Criação de pedidos por clientes.

    - Associação de itens de pedido com produtos.
    - Cálculo do valor total do pedido.
    - Atualização de status (por exemplo: CRIADO → CONFIRMADO → EM*PREPARO* → *SAIU*PARA_ENTREGA → ENTREGUE).
    - Controle de quem pode alterar o status (restaurante, entregador, admin).

3.3 Cache e Performance

- Uso de **Spring Cache**:
- Configurável por perfil (cache em memória no `dev` e `test`, Redis no `prod`).
- Caching de consultas frequentes (como listagem de cardápio) para reduzir latência.
- Integração com **Redis**:
- Repositório central de cache no perfil `prod`.
- Configuração via `spring-data-redis`.

3.4 Observabilidade, Monitoramento e Tracing

- **Spring Boot Actuator**:
    - Endpoints de health, info, métricas e outros.
    - Exposição controlada por perfil (mais aberta em `dev`, mais restrita em `prod`).
- **Micrometer & Tracing**:
    - Coleta de métricas técnicas (tempo de resposta, contadores de chamadas, etc.).
    - Traços distribuídos com Micrometer Tracing, Brave e Zipkin.
- **Zipkin**:
    - Recebimento e visualização de spans de tracing.
    - Análise de latência de chamadas e gargalos de performance.

3.5 Documentação da API

- Integração com **SpringDoc OpenAPI**:
- Geração automática de documentação OpenAPI 3.
- Interface gráfica via **Swagger UI** para exploração interativa.
- Configuração para exibir esquema de autenticação Bearer JWT.

3.6 Ambientes e Perfis

- **Dev**
    - Banco **H2 em memória**.
    - Console H2 disponível.
    - Cache simples em memória.
    - Logs detalhados (incluindo SQL e segurança).
    - Perfis ativos configurados em `application.yml`.
- **Test**
    - Banco H2 isolado para testes.
    - Cache desativado para evitar interferência nos testes.
    - Porta aleatória para evitar conflitos locais.

    - Segredos simplificados e previsíveis para JWT.
- **Prod**
    - Banco relacional (sugerido: PostgreSQL).
    - Redis para cache.
    - JWT configurado por variáveis de ambiente.
    - Logging mais enxuto, focado em operação.
    - Exposição controlada dos endpoints de Actuator.

4. Arquitetura do Sistema

4.1 Visão Geral da Arquitetura

A DeliveryTech API segue uma **arquitetura em camadas** com separação clara de responsabilidades:

- **Camada de Apresentação (Controller)**
    - Exposição dos endpoints REST.
    - Recebimento de requisições HTTP e devolução de respostas padronizadas.
    - Uso de DTOs de request/response para isolar o domínio da interface pública.
- **Camada de Aplicação/Serviço**
    - Contém a lógica de orquestração de casos de uso.
    - Aplica regras de negócio, validações e coordena operações com o repositório.
    - Colabora com a camada de segurança para autorização contextual.
- **Camada de Domínio (Model)**
    - Entidades JPA que representam os conceitos centrais (Usuário, Cliente, Restaurante, Produto, Pedido, etc.).
    - Modela relacionamentos e restrições.
- **Camada de Infraestrutura (Repository, Security, Config)**
    - Integração com o banco de dados via Spring Data JPA.
    - Configurações de segurança (JWT, filtros, CORS).
    - Configurações de cache, observabilidade e mapeamento OpenAPI.

Essa abordagem favorece **coesão interna**, **baixo acoplamento** e facilita a manutenção, evolução e testes do sistema.

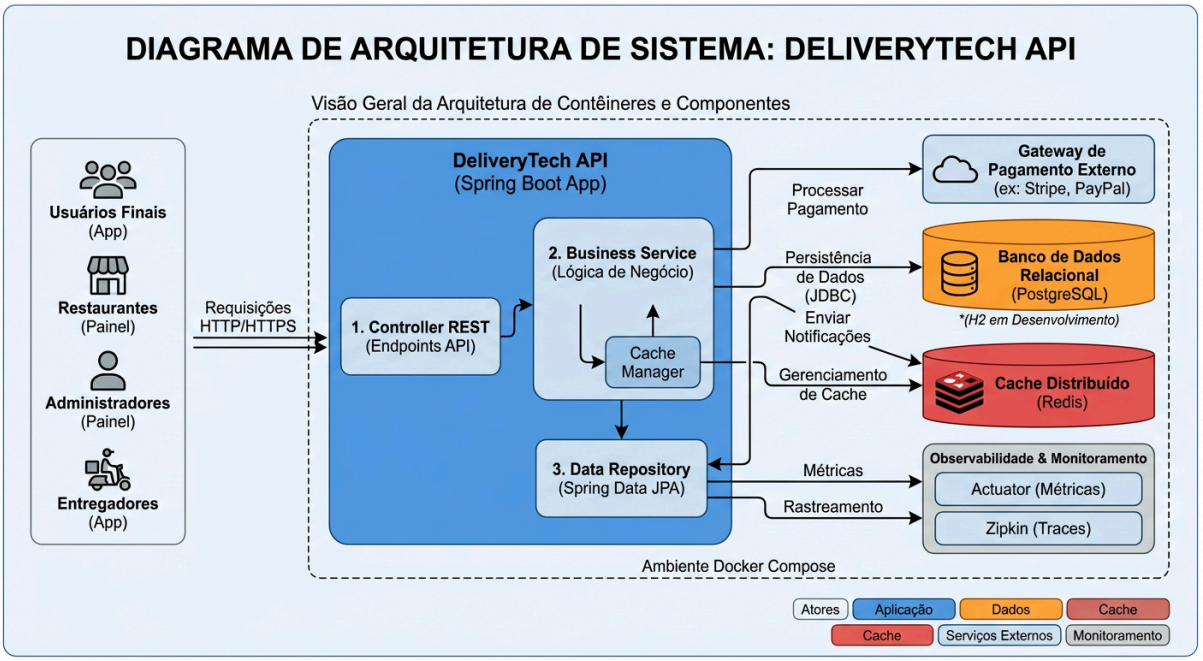


DIAGRAMA DE ARQUITETURA DE SISTEMA: DELIVERYTECH API
Visão Geral da Arquitetura de Contêineres e Componentes
Usuários Finais
(App)
Restaurantes
(Painel)
Administradores
(Painel)
Entregadores
(App)
Requisições
HTTP/HTTPS
DeliveryTech API
(Spring Boot App)
1. Controller REST
(Endpoints API)
2. Business Service
(Lógica de Negócio)
Cache
Manager
3. Data Repository
(Spring Data JPA)
Processar
Pagamento
Persistência
de Dados
(JDBC)
Enviar
Notificações
Gerenciamento
de Cache
Métricas
Rastreamento
Gateway de
Pagamento Externo
(ex: Stripe, PayPal)
Banco de Dados
Relacional
(PostgreSQL)
*(H2 em Desenvolvimento)
Cache Distribuído
(Redis)
Observabilidade & Monitoramento
Actuator (Métricas)
Zipkin (Traces)
Ambiente Docker Compose
Atores
Aplicação
Dados
Cache
Cache
Serviços Externos
Monitoramento

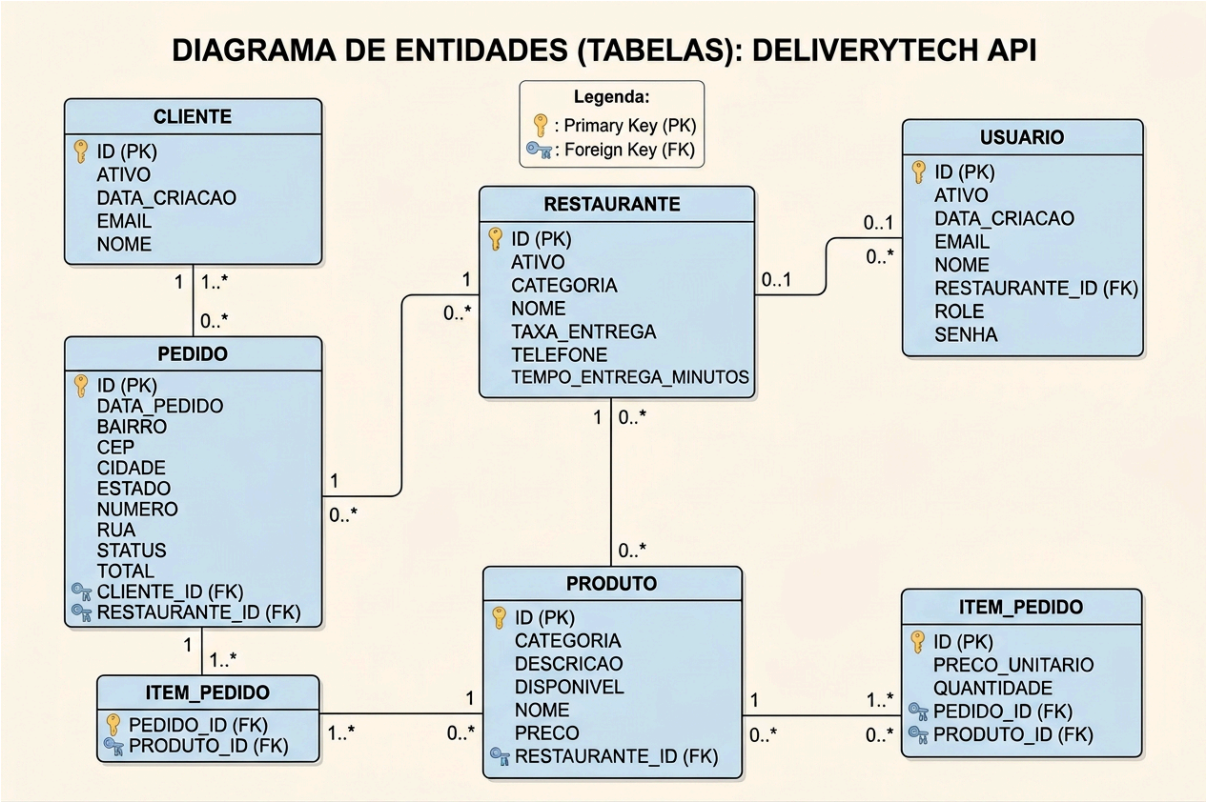
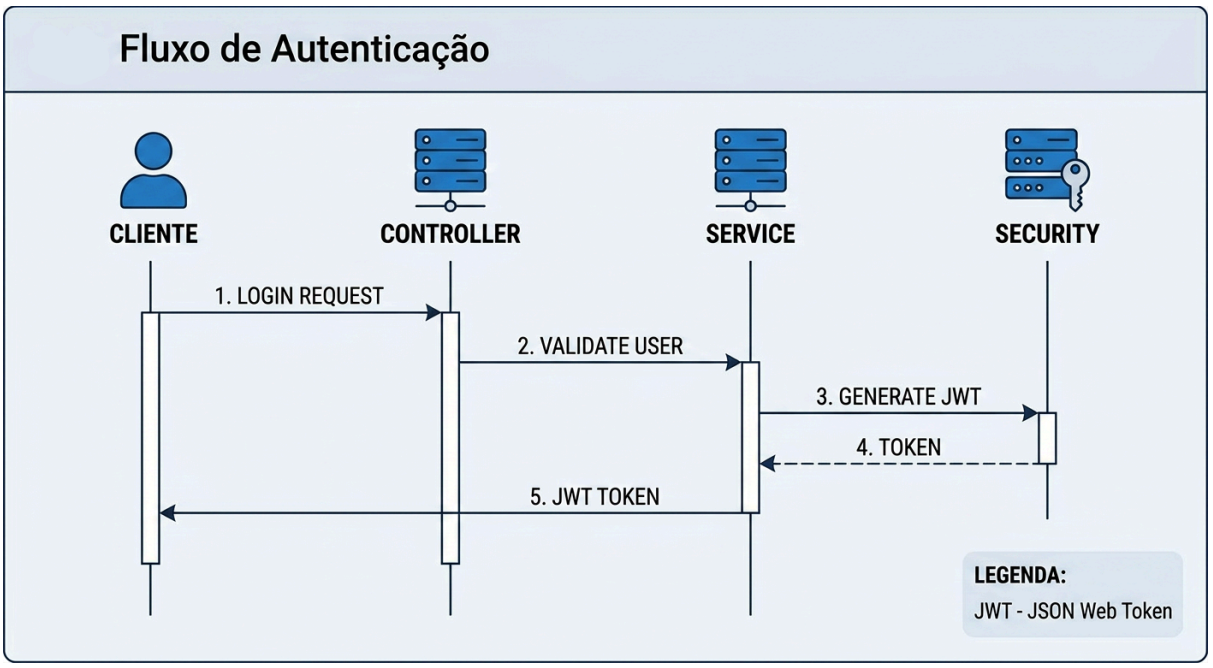


DIAGRAMA DE ENTIDADES (TABELAS): DELIVERYTECH API
Legenda:
: Primary Key (PK)
: Foreign Key (FK)
CLIENTE
ID (PK)
ATIVO
DATA_CRIACAO
EMAIL
NOME
PEDIDO
ID (PK)
DATA_PEDIDO
BAIRRO
CEP
CIDADE
ESTADO
NUMERO
RUA
STATUS
TOTAL
CLIENTE_ID (FK)
RESTAURANTE_ID (FK)
ITEM_PEDIDO
PEDIDO_ID (FK)
PRODUTO_ID (FK)
RESTAURANTE
ID (PK)
ATIVO
CATEGORIA
NOME
TAXA_ENTREGA
TELEFONE
TEMPO_ENTREGA_MINUTOS
USUARIO
ID (PK)
ATIVO
DATA_CRIACAO
EMAIL
NOME
RESTAURANTE_ID (FK)
ROLE
SENHA
PRODUTO
ID (PK)
CATEGORIA
DESCRICAO
DISPONIVEL
NOME
PRECO
RESTAURANTE_ID (FK)
ITEM_PEDIDO
ID (PK)
PRECO_UNITARIO
QUANTIDADE
PEDIDO_ID (FK)
PRODUTO_ID (FK)



4.2 Estrutura de Pastas

A estrutura principal do projeto é:

- `.github/`
- Workflows de CI/CD (GitHub Actions).
- `src/main/java/com/deliverytech/`
- `config/` – Configurações técnicas e integrações.
- `controller/` – Camada REST (endpoints públicos).
- `dto/`
    - `request/` – Objetos de entrada da API.
    - `response/` – Objetos de saída da API.
- `exception/` – Exceções customizadas e tratamento global.
- `model/` – Entidades JPA e modelos de domínio.
- `repository/` – Interfaces Spring Data JPA para acesso a dados.
- `security/` – Infraestrutura de JWT e integração com Spring Security.
- `service/` – Serviços de negócio.
- `DeliveryTechApiApplication.java` – Classe principal do Spring Boot.
- `src/main/resources/`
- `application.yml` – Configuração base e perfil ativo.
- `application-dev.yml` – Configuração para ambiente de desenvolvimento.
- `application-prod.yml` – Configuração para ambiente de produção.
- `application-test.yml` – Configuração para testes automatizados.
- `data.sql` (opcional) – Seed de dados para desenvolvimento.
- `src/test/java/com/deliverytech/`

- Testes de controllers, services e repositories.
- `Dockerfile` – Build da imagem Docker da API.
- `docker-compose.yml` – Orquestração de API, Redis, banco relacional e Zipkin.
- `pom.xml` – Configuração de build e dependências Maven.

4.3 Componentes Principais

- **Controllers**
  - `AuthController` – Endpoints de registro e login.
  - `ClienteController`, `RestauranteController`, `ProdutoController`, `PedidoController` – CRUDs e operações específicas.
- **Services**
- Serviços responsáveis por:

  - Validação de regras de negócio (por exemplo, se o restaurante está ativo para aceitar pedidos).
  - Cálculo de valores de pedido.
  - Alteração de status conforme o fluxo permitido.
  - Interação com cache para melhorar performance.
- **Repositories**
  - `UsuarioRepository`, `ClienteRepository`, `RestauranteRepository`, `ProdutoRepository`, `PedidoRepository` – Interfaces que abstraem o acesso a dados e deixam o código independente de detalhes de implementação.
- **Security**
  - `JwtTokenProvider` – Geração e validação de tokens.
  - `JwtAuthenticationFilter` – Extração de token do cabeçalho `Authorization` e autenticação do contexto.
  - `UserDetailsServiceImpl` – Carregamento de dados de usuário para o Spring Security.
  - `SecurityConfig` – Definição de rotas públicas, rotas protegidas e políticas de CORS.
- **Config**
  - `OpenApiConfig` – Metadados da documentação (título, descrição, segurança).
  - `CacheConfig` – Políticas de cache para cada perfil.
  - Configurações de observabilidade (Actuator, Micrometer, Zipkin).

5. Tecnologias e Bibliotecas Utilizadas

5.1 Backend Core

- **Java 21**
  - Última versão LTS da linguagem.
  - Melhorias de performance e recursos modernos.

- **Spring Boot 3.2.5**
  - Framework de aplicação que simplifica configuração e inicialização.
  - Suporte nativo a observabilidade, segurança e integração com diversas bibliotecas.

5.2 Web, Segurança e Validação

- **spring-boot-starter-web**
  - Servidor Tomcat embutido.
  - Suporte a JSON via Jackson.
  - Desenvolvimento de APIs REST.
- **spring-boot-starter-security**
  - Mecanismos de autenticação e autorização.
  - Filtros e integração com contexto de segurança.
- **spring-boot-starter-validation**
  - Validação de campos de entrada com Bean Validation (anotações como `@NotNull`, `@Email`, etc.).
- **JJWT (io.jsonwebtoken)**
  - `jjwt-api`, `jjwt-impl`, `jjwt-jackson`.
  - Criação, assinatura e parsing de JSON Web Tokens (JWT).

5.3 Persistência, Banco de Dados e Cache

- **spring-boot-starter-data-jpa**
  - ORM com Hibernate.
  - CRUD simplificado com repositórios Spring Data.
  - Mapeamento objeto-relacional das entidades.
- **H2 Database**
  - Banco em memória utilizado em `dev` e `test`.
  - Rápido, leve e de fácil configuração.
- **spring-boot-starter-cache**
  - Anotações para gerenciamento de cache em nível de método.
  - Abstração de implementação de cache (Simple, Redis, etc.).
- **spring-data-redis**
  - Conector com Redis.
  - Utilizado no perfil de produção para cache distribuído.

5.4 Observabilidade e Monitoramento

- **spring-boot-starter-actuator**
  - Endpoints para saúde, métricas e informações da aplicação.
  - Integração com ferramentas de monitoramento externas.
- **micrometer-observation**
  - Métricas e observabilidade de forma unificada.
- **micrometer-tracing-bridge-brave**
  - Integra a API de tracing do Micrometer com o Brave (tracer popular para Zipkin).

- **zipkin-reporter-brave**
  - Envio de spans e traces para o servidor Zipkin.

5.5 Documentação

- **springdoc-openapi-starter-webmvc-ui**
  - Geração automática da especificação OpenAPI.
  - Interface Swagger UI na rota `/swagger-ui.html`.

5.6 Testes

- **spring-boot-starter-test**
  - JUnit 5, AssertJ, Hamcrest e utilitários para testes com Spring.
- **mockito-core**
  - Criação de mocks para isolamento de unidades.
- **spring-security-test**
  - Ferramentas para testar endpoints protegidos com segurança.

5.7 Utilitários

- **Lombok**
  - Redução de código repetitivo com anotações como `@Data`, `@Builder`, `@Slf4j`.
- **Docker e Docker Compose**
  - Empacotamento da aplicação em containers.
  - Orquestração de API, banco de dados e Redis.

6. Perfis de Configuração e Ambientes

6.1 Arquivo Base (`application.yml`)

- Define:
  - Nome da aplicação.
  - Perfil ativo padrão (`dev`).
  - Configurações comuns a todos os ambientes (por exemplo, paths do Swagger, propriedades de tracing).

6.2 Perfil de Desenvolvimento (`application-dev.yml`)

- Banco H2 em memória.
- `ddl-auto: create-drop` para facilitar a evolução de schema durante desenvolvimento.
- Console H2 habilitado.
- Cache do tipo `simple` (em memória).
- Logging detalhado:
- SQL formatado.

- Logs de segurança.
- Logs de pacotes da aplicação.

6.3 Perfil de Produção (`application-prod.yml`)

- Banco relacional externo (recomendado: PostgreSQL).
- Pool de conexões configurado via HikariCP.
- `ddl-auto: validate` para não alterar o schema automaticamente em produção.
- Cache do tipo `redis`.
- Configuração de host/porta/credenciais de Redis por variáveis de ambiente.
- Logging mais enxuto e orientado à operação.
- Endpoints de Actuator expostos de forma mais restrita.
- Segredos (como `JWT_SECRET`) obrigatoriamente fornecidos via variáveis de ambiente.

6.4 Perfil de Testes (`application-test.yml`)

- Banco H2 dedicado aos testes automatizados.
- `ddl-auto: create-drop` para garantir isolamento entre execuções.
- Cache desativado para evitar interferência nos resultados.
- Porta aleatória para não conflitar com ambientes locais.
- Segredo JWT fixo e simples, adequado ao contexto controlado de testes.

7. Fluxos Principais

7.1 Fluxo de Autenticação com JWT

1. **Registro**
    - O cliente envia um `POST /api/auth/register` com dados de usuário.
    - O sistema valida os dados e persiste a entidade `Usuario` associada a um papel (role).
1. **Login**
    - O cliente envia um `POST /api/auth/login` com email e senha.
    - As credenciais são validadas pelo Spring Security.
    - Em caso de sucesso, é gerado um token JWT contendo:
        - Identificador do usuário.
        - Roles (perfis de acesso).
        - Data de expiração.
    - O token é devolvido ao cliente no corpo da resposta.
1. **Autorização**
    - Nas chamadas subsequentes, o cliente envia o token no header `Authorization: Bearer <token>`.
    - Um filtro (`JwtAuthenticationFilter`) intercepta a requisição:
        - Extrai e valida o token.
        - Carrega o usuário e roles.

        - Popula o contexto de segurança.
    - Controladores e métodos são protegidos com base nos perfis necessários.

7.2 Fluxo de Criação e Processamento de Pedidos

1. O cliente autenticado envia um `POST /api/pedidos` com:
    - Identificação do restaurante.
    - Lista de itens de pedido (produto, quantidade).
1. A camada de serviço:
    - Valida se o restaurante está ativo.
    - Valida se os produtos pertencem ao restaurante e estão disponíveis.
    - Calcula os valores (subtotal, total).
    - Define o status inicial do pedido.
1. O pedido é persistido no banco de dados e o identificador retornado ao cliente.
2. Perfis de restaurante e entregador usam endpoints específicos para:
    - Atualizar status do pedido.
    - Acompanhar pedidos em preparação ou em rota.

8. Estratégia de Cache e Desempenho

8.1 Objetivo do Cache

Reduzir a latência e a carga sobre o banco de dados em consultas altamente frequentes, especialmente:

- Listagem de cardápios (produtos por restaurante).
- Consultas repetidas de pedidos com pouca variação.

8.2 Implementação

- Uso de anotações como `@Cacheable`, `@CacheEvict` e `@CachePut` nos serviços.
- Em desenvolvimento:
- `cache.type=simple`, implementado na memória da própria JVM.
- Em produção:
- Integração com Redis via `spring-data-redis`.
- Redis configurado como cache store central.

8.3 Benefícios Observados (Metas de Projeto)

- Redução significativa de tempo de resposta em endpoints cacheados.
- Menor uso de CPU e I/O de disco do banco de dados.
- Melhor escalabilidade horizontal quando múltiplas instâncias da API compartilham o mesmo Redis.

9. Observabilidade, Monitoramento e Métricas

9.1 Actuator

- Endpoints como `/actuator/health` e `/actuator/metrics` permitem:
- Verificar disponibilidade da aplicação.
- Monitorar métricas técnicas básicas (tempo de resposta, uso de memória, etc.).

9.2 Micrometer e Zipkin

- **Micrometer** integra métricas e tracing com bibliotecas de monitoramento.
- **Tracing distribuído**:
- Cada requisição gera spans que representam operações internas (chamadas a serviços, acesso a repositórios).
- Os spans são enviados ao **Zipkin**, que oferece uma interface para análise:
    - Tempo total da requisição.
    - Quebras de tempo por camada.
    - Detecção de gargalos.

10. Estrutura de Testes

10.1 Estratégia

- Separação entre testes unitários e testes de integração:
- **Unitários**:
    - Focados em serviços e componentes isolados.
    - Uso intenso de Mockito.
- **Integração**:
    - Verificam o comportamento end-to-end de controllers, segurança e persistência.
    - Uso de Spring Boot Test, MockMvc e Spring Security Test.

10.2 Recursos Utilizados

- Perfil `test` garante:
- Banco isolado.
- Ausência de cache para previsibilidade.
- Configurações de logs e portas adequadas.

11. Desafios e Soluções

11.1 Gestão de Cache e Consistência

Desafio
Garantir alta performance para consultas frequentes (como cardápios), sem comprometer a consistência dos dados.

Solução

- Uso de **Spring Cache** para cachear respostas de consultas mais acessadas.
- Invalidação e atualização do cache em operações de escrita (por exemplo, ao atualizar ou remover produto).
- Redis em produção para permitir que múltiplas instâncias da API compartilhem o mesmo cache.

11.2 Segurança e Controle de Acesso

Desafio
Implementar autenticação e autorização robustas com controle de acesso granular de acordo com o perfil do usuário.

Solução

- Integração do Spring Security com JWT.
- Definição de roles por tipo de usuário.
- Proteção de endpoints por role, garantindo que:
- Apenas clientes possam criar pedidos.
- Apenas restaurantes e entregadores possam alterar o status de pedidos em certas etapas.
- Apenas administradores possam acessar endpoints de gestão global.

11.3 Performance e Escalabilidade

Desafio
Manter tempos de resposta baixos e garantir que a solução possa crescer para suportar maior volume de requisições.

Solução

- Otimização de queries JPA e mapeamentos.
- Uso de cache multi-camadas (memória no dev, Redis no prod).
- Configuração de pool de conexões no banco de dados em produção.
- Observabilidade e métricas para identificar gargalos.

11.4 Organização Arquitetural

Desafio
Manter o código organizado e de fácil expansão à medida que novas funcionalidades são adicionadas.

Solução

- Adoção de uma arquitetura em camadas clara (controller, service, repository, model, security, config).
- Uso de DTOs para desacoplar a API pública do modelo de domínio.
- Padronização de tratamento de erros e respostas através de um handler global de exceções.

12. Possíveis Melhorias Futuras

12.1 Melhorias Técnicas

- Migração definitiva para **PostgreSQL** em ambientes de produção.
- Introdução de mensageria com **RabbitMQ** ou **Kafka** para:
- Processamento assíncrono de eventos (ex.: envio de notificações).
- Desacoplamento de serviços futuros.
- Ampliação da observabilidade:
- Integração com **Prometheus** e **Grafana** para dashboards de métricas.
- Aumento da cobertura de testes:
- Testes de integração mais abrangentes.
- Testes de contrato para garantir compatibilidade de APIs.

12.2 Melhorias Funcionais

- Sistema de avaliações de pedidos e restaurantes.
- Integração com serviços de mapas para cálculo de rotas e estimativa de entrega.
- Sistema de fidelidade, com pontos e recompensas para clientes.
- Gestão de promoções, cupons e descontos.
- Suporte a múltiplos meios de pagamento e integração com gateways externos.

13. Métricas de Sucesso

Para avaliar a evolução do projeto, algumas metas recomendadas:

- **Performance**
- Tempo médio de resposta (p95) abaixo de 100 ms para operações de leitura.
- **Qualidade**
- Cobertura de testes acima de 80% nas camadas de serviço e controle.
- **Disponibilidade**
- Disponibilidade da API acima de 99,9% (em cenários com múltiplas instâncias).
- **Segurança**
- Zero vulnerabilidades críticas identificadas por ferramentas de análise estática e testes de segurança.

14. Conclusão

A DeliveryTech API entrega uma base sólida para sistemas de delivery modernos, combinando **boas práticas de arquitetura**, **segurança robusta**, **observabilidade** e **performance**. O uso de tecnologias atuais do ecossistema Java/Spring aliado à containerização e automação com Maven e Docker fornece um ambiente propício tanto para aprendizado quanto para evolução em direção a uma solução de produção.

O projeto permanece aberto à extensão, com pontos claros para inclusão de novas funcionalidades, integrações externas e melhorias contínuas de qualidade técnica e funcional.